

Analog IC Design (Xschem, Ngspice, ADT)**Lab 05****Simple vs Wide Swing (Low Compliance) Cascode Current Mirror****Intended Learning Objectives**

In this lab you will:

- Explore current mirror sizing trade-offs using Sizing Assistant (SA).
- Bias a cascode device using a series resistance.
- Design and simulate simple and wide swing (low-voltage) current mirrors.
- Compare simple and wide swing current mirrors.
- Investigate the effect of mismatch on a wide swing current mirror.

Part 1: Exploring Sizing Tradeoffs Using SA

- 1) We want to design a simple current mirror with the following specs.

Parameter	
Current direction (source/sink)	Sink
Input Current	$10\mu A$
Output Current	$20\mu A$
% Change in Current for $\Delta V_{out} = 1V$	$< 10\%$
Percent mismatch: $\sigma(I_{out})/I_{out}$	$\leq 2\%$
Compliance voltage	$\leq 150mV$
Area	Minimize

- 2) Sinking current means which device type? NMOS or PMOS?
- 3) The % Change in current translates to a spec on the $\lambda = 1/V_A$ of the device. How much is the required λ ?
- Note:** λ and V_A significantly depend on V_{DS} . Assume that the λ spec is required at $V_{DS} = 0.5V$.
- 4) The current mirror bias point trade-offs are summarized in the table below (**assuming a fixed total area**).

Note: For a given gate area, g_m is roughly constant. Given an error voltage at the gate, a smaller g_m results in a smaller error current. That's why a smaller g_m/I_D (a larger V_{star}) is usually desirable for a current mirror.

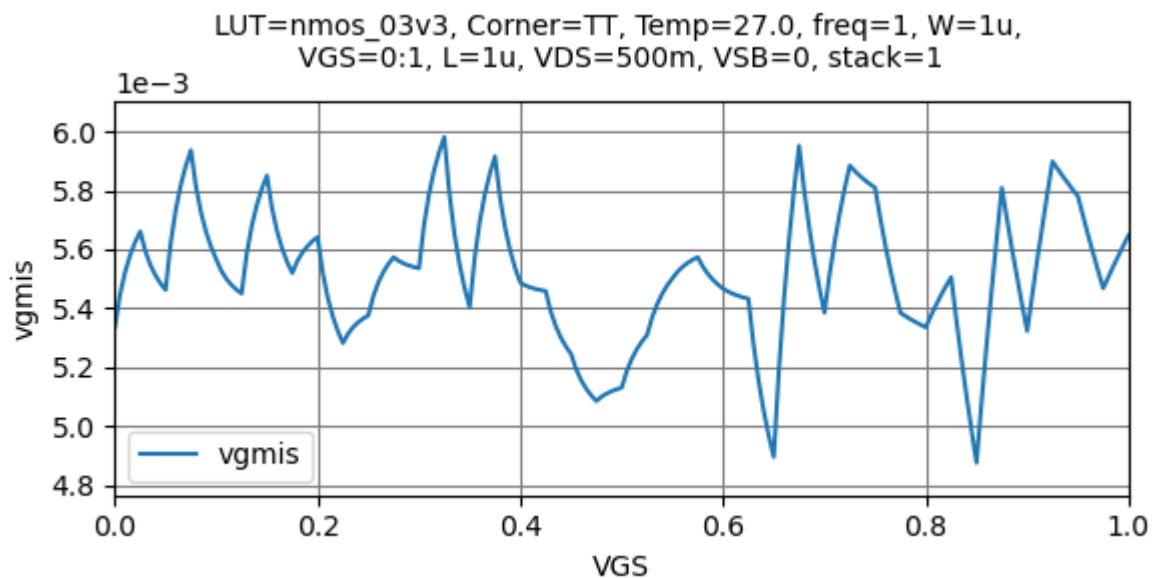
Note: For a fixed total area, when g_m/I_D increases (W increases) then L will decrease and vice versa.

Parameter	Higher gm/ID (lower V^*)	Lower gm/ID (higher V^*)
Dependence on V_{DS} (output resistance, $\lambda = 1/V_A$)	☹️	😊
Random mismatch	☹️	😊
Systematic mismatch	☹️	😊
Compliance voltage (headroom)	😊	☹️

- 5) The most important mismatch effect is the VTH mismatch, which is modeled by Pelgrom's equation:

$$\sigma_{VT} = \frac{A_{VT}}{\sqrt{W \times L}}$$

Set a device that has $W \times L = 1 \mu\text{m}^2$ so that $\sigma_{VT} = A_{VT}$. Plot vgmis vs VGS in SA. Note the random variations because vgmis is extracted from a Monte Carlo (MC) simulation that has finite no. of samples (finite population), so this is an estimate of the standard deviation, not the true one. If the mismatch is characterized using dcmatch simulation (not always supported by the model), the results will be smooth and more accurate.



- 6) From the model file, find σ_{VT} (var_vth at $W \times L = 1 \mu\text{m}^2$) and compare it to the value in ADT. Which one is higher, why?

Note: par_vth is the mismatch between two identical transistors, which is equal to $\sqrt{2}$ times the mismatch variation of one transistor, i.e., the mismatch error of one transistor = $\text{par_vth} / \sqrt{2}$. Note that when we add two uncorrelated random variables, we add the variance, not the standard deviation.

[gedit ~/pdk/gf180mcuC/libs.tech/ngspice/sm141064.ngspice &](#)

```

46983 .lib fets_mm
46984 .subckt nmos_3p3 d g s b w=1e-5 l=2.8e-7
46985 + as=0 ad=0 ps=0 pd=0 nrd=0 nrs=0 par=1 dtemp=0
46986 + sa=0 sb=0 nf=1 sd=0 m=1
46987
46988 |.param
46989 + par_vth=0.007148
46990 + par_k=0.007008
46991 + par_l=1.5e-7
46992 + par_w=-1e-7
46993 + par_leff='l-par_l'
46994 + par_weff='par*(w-par_w)'
46995 + p_sqrtarea='sqrt((par_leff)*(par_weff))'
46996
46997 .param
46998 + var_k='0.7071*par_k* 1e-06 / p_sqrtarea'
46999 + mis_k=agauss(0,var_k,1)
47000
47001 .param
47002 + var_vth='0.7071*par_vth* 1e-06 / p_sqrtarea'
47003 + mis_vth=agauss(0,var_vth,1)
47004
47005 m0 d g s b nmos_3p3 w=w l=l as=as ad=ad ps=ps pd=pd nrd=nrd nrs=nrs
47006 +delvto='mis_vth*sw_stat_mismatch' sa=sa sb=sb nf=nf sd=sd
47007 .ends nmos_3p3
47008 *-----

```

- 7) Examine these trade-offs using SA. Use SA to plot the sizing at a constant $\sigma(I_{out})/I_{out}$ which is given by

$$\frac{\sigma(I_{out})}{I_{out}} = \frac{\sqrt{1 + \frac{1}{m}} \times idmis}{I_D} \times 100$$

The $\sqrt{1 + \frac{1}{m}}$ factor is due to taking into account the effect of two random variables (V_{TH} of the two current mirror transistors).

Note: idmis and I_D are those of the 10uA device (the reference device) and the 20uA device (the mirror output device) variance is $idmis^2/m$, where $m=2$ because it has twice the area. Note that when we add two uncorrelated random variables, we add the variance, not the standard deviation.

Plot L, W, AREA, and λ at the settings shown below. The results will not be smooth due to the noisy MC mismatch data in the LUT.

ID	10u
Vstar	100m:200m
$\sqrt{1.5} \times idmis / ID \times 100$	1, 2
VDS	0.5
VSB	0
Stack	1

- 8) Does the results indicate that the highest V^* is desirable from the perspective of mismatch, area, and λ ? If yes, then the maximum V^* will be limited by the required compliance voltage.
Note: We assume that the compliance voltage $\approx V_{Dssat} \approx V^*$.
- 9) **Report** the above plots with a cursor added at the required V^* . Does this point satisfy the mismatch and λ constraints?

10) If the λ constraint is not satisfied at $\sigma(I_{out})/I_{out} = 2\%$, i.e., it needs a longer L , we can use SA to find the required design point as shown below.

Note: Lambda is defined in ADT expression manager.

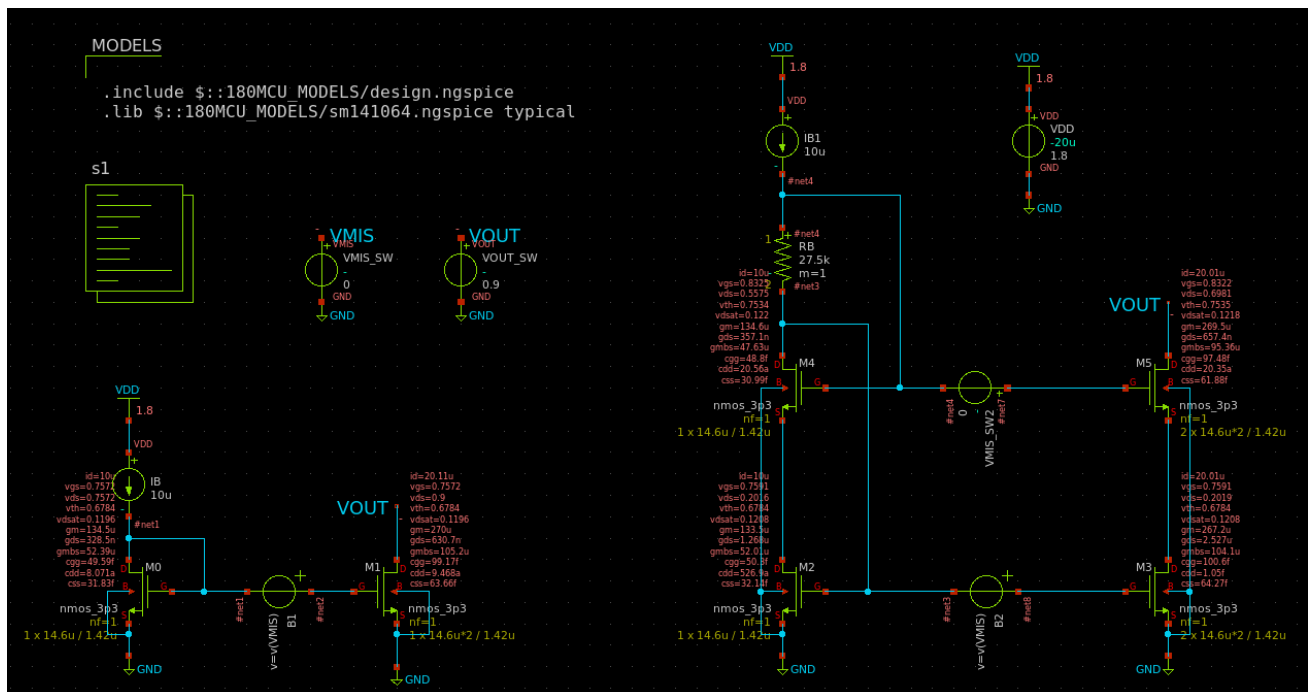
Note: We should pick the longer L such that the two required conditions (lambda and mismatch) are satisfied.

ID	10u
Vstar	150m
LAMBDA	0.1
VDS	0.5
VSB	0
Stack	1

11) **Report** the device sizing, λ and $\sigma(I_{out})/I_{out}$ at the selected design point.

Part 2: Current Mirror Simulation

1) Create a new schematic. Construct the circuit shown below.



- 2) We use a single vsource to define the output of both circuits so we can sweep their VOUT simultaneously.
- 3) We use two "asrc" instances (B1 and B2) to define a used-added mismatch for the two circuits. When VMIS_SW is swept, this will affect both B1 and B2 because they are defined as equations of the voltage at VMIS net as shown below. VMIS_SW2 is defined as a normal vsource because it is used only in one place in the schematic.

- 4) The current mirror takes input current I_B and generates output current $= 2 \cdot I_B$. The correct way to make the mirroring ratio $= 2$ is to use the multiplier parameter¹ or to use an array of parallel devices, e.g., M1[1..2]². In this lab we will simply multiply W by 2, but remember that in a real design you must use multiples of identical unit elements and you should never just multiply W by 2.
- 5) Instead of using a wide-swing bias transistor (a magic battery) to generate V_B , we use a resistor R_B in series with the input branch.
- 6) Unless otherwise stated, set $V_{OUT} = V_{DD}/2$ and $V_{MIS_SW} = V_{MIS_SW2} = 0$.

1. Design and OP (Operating Point) Analysis

- 1) Assume we want to set a $50mV$ saturation margin for M2 and M3, i.e., $V_{DS2} \approx V_{DS3} \approx V^* + 50mV$. Ignore the body effect and **calculate** a rough value for R_B .
Hint: $R_B = \frac{V_{GS4} + V_{DS2} - V_{GS2}}{I_B} \approx \frac{V_{DS2}}{I_B}$
Hint: The purpose of doing rough analysis is not to reach a final design point, but to calculate a value that makes sense and can be used to determine a reasonable range for a simulator sweep.
- 2) Perform DC sweep (not parametric sweep) for R_B . Choose a reasonable sweep range given the rough value computed in the previous step. **Report** V_{DS3} vs R_B . **Choose** R_B to satisfy the $50mV$ saturation margin requirement. Is the selected R_B value larger or smaller than the rough analytical value? Why?
 - ➔ Hint: The DC sweep is performed in a simulator inner loop, so it is very fast and takes small disk space. The parametric sweep is an outer loop repetitive calling of the simulator, so it is much slower and takes much larger disk space.
 - ➔ Hint: In Ngspice, DC sweep can be performed only for a voltage source, a current source, or a resistance. A parameter cannot be used in a simple DC sweep.
- 3) Simulate the OP point. Report a snapshot clearly showing the following parameters for each transistor.

ID
VGS
VDS
VTH
VDSAT
GM
GDS
GMB

- 4) Do all transistors operate in saturation?

➔ **Example ngspice control block:**

```
.control
```

¹ The multiplier (m) parameter of the MOSFET is ignored due to a bug in some versions of the PDK and was fixed later as reported [here](#).

² http://repo.hu/projects/xschem/xschem_man/tutorial_busses.html

```

alterparam sw_stat_global = 0
alterparam sw_stat_mismatch = 0
reset

save all

let x = 0
set num = {$&x}
dowhile x <= 5
    save @m.xm{$num}.m0[id]
    save @m.xm{$num}.m0[vgs]
    save @m.xm{$num}.m0[vds]
    save @m.xm{$num}.m0[vdsat]
    save @m.xm{$num}.m0[vth]
    save @m.xm{$num}.m0[gm]
    save @m.xm{$num}.m0[gds]
    save @m.xm{$num}.m0[gmbbs]
    save @m.xm{$num}.m0[cgg]
    save @m.xm{$num}.m0[cdd]
    save @m.xm{$num}.m0[css]
    let x = x + 1
    set num = {$&x}
end
op
write cmirror_op.raw
set appendwrite
dc RB 20k 50k 100
remzerovec
write cmirror_op.raw
.endc

```

2. DC Sweep (I_{out} vs V_{OUT})

- 1) Perform DC sweep (not parametric sweep) using $V_{OUT} = 0:10m:V_{DD}$. Report I_{out} vs V_{OUT} for the two CMs overlaid in the same plot.
 - Comment on the difference between the two circuits.
 - From the plot, find an estimate for the compliance voltage of each current mirror.
 - I_{out} of the simple CM is exactly equal to $I_B \cdot 2$ at a specific value of V_{OUT} . Why?
- 2) Calculate the percent change in I_{out} when V_{OUT} changes from 0.5V to 1.5V (i.e., 1V change).
 - Compare the simple mirror result to the value expected from Part 1.
 - Comment on the difference between the two circuits.
- 3) Report the percent of error in I_{out} vs V_{OUT} (ideal I_{out} should be $I_B \cdot 2$) for the two CMs in the current mirror operating region (zoom in the region $V_{OUT} \approx V^*$ to V_{DD}) overlaid in the same plot.
 Hint: Calculate percent of error as $(\text{simulated} - \text{ideal}) / \text{ideal} \cdot 100$
 - Comment on the difference between the two circuits.
- 4) Report R_{out} vs V_{OUT} (take the inverse of the derivative of I_{out} plot) for the two CMs in the current mirror operating region ($V_{OUT} \approx V^*$ to V_{DD}) overlaid in the same plot. Use log scale on the y-axis. Add a cursor at $V_{OUT} = V_{DD}/2$.
 - Comment on the difference between the two circuits.

- Does Rout change with VOUT? Why?

➔ Hint: Rout can also be simulated using AC analysis. The value we calculated here should be similar to the AC analysis result at low frequencies.

- 5) Analytically calculate Rout of both circuits at VOUT = VDD/2. Compare with simulation results in a table.

➔ **Example ngspice control block:**

```
.control

alterparam sw_stat_global = 0
alterparam sw_stat_mismatch = 0
reset

save all

let x = 0
set num = {$&x}
dowhile x <= 5
    save @m.xm{$num}.m0[id]
    save @m.xm{$num}.m0[vgs]
    save @m.xm{$num}.m0[vds]
    save @m.xm{$num}.m0[vdsat]
    save @m.xm{$num}.m0[vth]
    save @m.xm{$num}.m0[gm]
    save @m.xm{$num}.m0[gds]
    save @m.xm{$num}.m0[gmbs]
    save @m.xm{$num}.m0[cgg]
    save @m.xm{$num}.m0[cdd]
    save @m.xm{$num}.m0[css]
    let x = x + 1
    set num = {$&x}
end

save all

* tighten the tolerances to improve the accuracy
*option reltol=1e-5 vntol=1e-6 abstol=1e-12

dc VOUT_SW 0 1.8 10m

echo \"Simple Current Mirror Simulation Results\"
meas dc i1 find @m.xm1.m0[id] when v(VOUT)=0.5V
meas dc i2 find @m.xm1.m0[id] when v(VOUT)=1.5V
let perc_change = (i2 - i1) / ((i2 + i1)/2) * 100
print perc_change

echo \"Wide Swing Current Mirror Simulation Results\"
meas dc i1 find @m.xm3.m0[id] when v(VOUT)=0.5V
meas dc i2 find @m.xm3.m0[id] when v(VOUT)=1.5V
let perc_change = (i2 - i1) / ((i2 + i1)/2) * 100
```

```

print perc_change

* Rout calculation
let Rout1 = 1/deriv(@m.xml.m0[id])
let Rout2 = 1/deriv(@m.xml.m0[id])

* Percent error calculation
let Error1 = (@m.xml.m0[id] - 20u) / 20u * 100
let Error2 = (@m.xml.m0[id] - 20u) / 20u * 100

write cmirror_dc.raw

.endc

```

3. Mismatch

NOTE: Usually we study the mismatch using Monte Carlo simulation as will be shown in the next section. However, in this section, we will manually add mismatch in the circuit.

- 1) Perform DC sweep for VMIS_SW from 0 to sqrt(1.5)*v_{gmis} (one-sigma variation) and set VMIS_SW2 = 0. This models the standard deviation of the mismatch in V_{TH} for the current mirror devices. Find the percent change in I_{out} .
- 2) Analytically calculate the percent change in I_{out} and compare it to the simulation result.
Hint: The voltage change at the gate can be considered as a small signal. Thus, the change in the current can be calculated using the G_m of the circuit. In this case, the circuit can be considered as a cascode amplifier.
- 3) Set VMIS_SW = 0 and perform DC sweep for VMIS_SW2 from 0 to sqrt(1.5)*v_{gmis}. This models the standard deviation of the mismatch in V_{TH} for the cascode devices. Find the percent change in I_{out} .
- 4) Analytically calculate the percent change in I_{out} and compare it to the simulation result.
Hint: The voltage change at the gate can be considered as a small signal. Thus, the change in the current can be calculated using the G_m of the circuit. In this case, the circuit can be considered as a **degenerated** common source amplifier.
- 5) Which mismatch contribution is more pronounced? Why?
- 6) For a given total area: Which design decision is better: setting the same W and L for the mirror and cascode devices? Or using larger W and L for the current mirror devices? Why?

➔ Example ngspice control block:

```

.control

alterparam sw_stat_global = 0
alterparam sw_stat_mismatch = 0
reset

save all

let x = 0
set num = {${x}}
dowhile x <= 5
  save @m.xml{$num}.m0[id]
  save @m.xml{$num}.m0[vgs]
  save @m.xml{$num}.m0[vds]

```



```

save @m.xml{$num}.m0[vdsat]
save @m.xml{$num}.m0[vth]
save @m.xml{$num}.m0[gm]
save @m.xml{$num}.m0[gds]
save @m.xml{$num}.m0[gmbs]
save @m.xml{$num}.m0[cgg]
save @m.xml{$num}.m0[cdd]
save @m.xml{$num}.m0[css]
let x = x + 1
set num = {$&x}
end

save all

* tighten the tolerances to improve the accuracy
*option reltol=1e-5 vntol=1e-6 abstol=1e-12

dc VMIS_SW 0 1.7m 0.1m

echo \"Simple Current Mirror Simulation Results\"
meas dc i1 min @m.xml1.m0[id]
meas dc i2 max @m.xml1.m0[id]
let perc_change = (i2 - i1) / ((i2 + i1)/2) * 100
print perc_change

echo \"Wide Swing Current Mirror Simulation Results\"
meas dc i1 min @m.xml3.m0[id]
meas dc i2 max @m.xml3.m0[id]
let perc_change = (i2 - i1) / ((i2 + i1)/2) * 100
print perc_change

write cmirror_dc.raw

dc VMIS_SW2 0 1.7m 0.1m

echo \"Wide Swing Current Mirror Simulation Results (Cascode
Mismatch)\"
meas dc i1 min @m.xml3.m0[id]
meas dc i2 max @m.xml3.m0[id]
let perc_change = (i2 - i1) / ((i2 + i1)/2) * 100
print perc_change

set appendwrite
write cmirror_dc.raw

.endc

```

4. Monte Carlo (MC) Simulation

Note: Enable Monte Carlo in the model file.

```
gedit ~/pdk/gf180mcuC/libs.tech/ngspice/design.ngspice &
```

```

22 ** MonteCarlo and matching simulation setting:
23 ** -----
24 ** sw_stat_global
25 ** sw_stat_mismatch
26 **
27 **
28 ** |          setting          | sw_stat_global=0 | sw_stat_global=1 |
29 ** |-----|-----|
30 ** | sw_stat_mismatch=0 | No statistical   | Global variation is on, |
31 ** |                   | modeling        | but mismatch is off.   |
32 ** |-----|-----|
33 ** | sw_stat_mismatch=1 | mismatch is on,  | Most realistic          |
34 ** |                   | global variation off | distribution.           |
35 ** |-----|-----|
36 **
37 **
38 ** (default) - sw_stat_global=1 and sw_stat_mismatch=1
39 ** This setting provides the most complete representation of the
40 ** statistical variations during chip manufacturing.
41 ** Global process variations are determined by random distributions.
42 ** Mismatch is differentiated from global variation in that mismatch only
43 ** includes intra-die variation, and it is especially critical for analog matching applications.
44 **
45 ** mc_skew is the monte-carlo simulation variation control.
46 **
47 **
48 ** -----
49 ** Flicker noise corner setting:
50 ** -----
51 **
52 ** "fnoicor" switch is added for user to select between the best- or worst-case
53 ** flicker noise simulation options
54 ** fnoicor = 0 : (default) as-extracted simulation
55 ** fnoicor = 1 : worst case simulation
56 **
57 ** *****
58 **
59 ** Switches
60 **
61 ***** Default mc switches *****
62 **
63 .param
64 + sw_stat_global = 1
65 + sw_stat_mismatch = 1

```

➔ **Note:** Instead of editing the model file, an easier way will be forcing these parameters to zero in the netlist itself by using the following commands in the beginning of the .control block:

```

alterparam sw_stat_global = 1
alterparam sw_stat_mismatch = 1
reset

```

➔ **Note:** In order to make the MC results reproducible, you can use this command:

```

.options seed=<value>

```

➔ **Note:** Using the multiplier parameter can give wrong mismatch results for the some model files because the errors from the parallel devices will be correlated. The safest approach is to set m = 1, and name the device as an array of two devices 'Mx[1..0]'. Check [this article](#) for more information.

➔ **Note:** In this section we will set both VMIS_SW and VMIS_SW2 to zero.

- 1) Set up MC simulation by using a loop similar to the code given below.

Note: Test and debug with 10 runs, then do the real simulation with 200 runs.

➔ **Example ngspice control block:**

```

.control

```

```

shell rm MC_WS_CM.csv
shell rm MC_Simple_CM.csv

alterparam sw_stat_global = 1
alterparam sw_stat_mismatch = 1
reset

let mc_runs = 10
let run = 1
dowhile run <= mc_runs
    op
    print @m.xml.m0[id] >> MC_Simple_CM.csv
    print @m.xml.m3[id] >> MC_WS_CM.csv
    reset
let run = run + 1
end

.endc

```

- 2) Use Excel or Matlab to plot the histogram of I_{out} .
- 3) Calculate the standard deviation percentage $\sigma(I_{out})/I_{out}$.
- 4) Compare the MC simulation result to the expected analytical result.

Lab Summary

In Part 1 you learned:

- How to use SA to examine current mirror design trade-offs.
- How to design a simple current mirror.

In Part 2 you learned:

- How to design a wide swing (low-voltage) current mirror.
- How the behavior of a current mirror changes with the output voltage.
- The effect of mismatch on a current mirror.
- How to perform Monte Carlo simulations for a current mirror.

Acknowledgements

Thanks to all who contributed to these labs. Special thanks to Dr. Sameh A. Ibrahim for reviewing and editing the labs. If you find any errors or have suggestions concerning these labs, please contact Hesham.omran@eng.asu.edu.eg.